RISC-V Instruction Set Summary (1)

Register (R-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R-type	funct7							rs2					rs1					funct3			rd					opcode						

opcode	funct3	funct7	instruction	description	operation
7'h33	3'b000	7'h00	add rd, rs1, rs2	add	rd = rs1 + rs2
7'h33	3'b000	7'h20	sub rd, rs1, rs2	subtract	rd = rs1 - rs2
7'h33	3'b001	7'h00	sll rd, rs1, rs2	shift left logical	rd = rs1 << $rs2_{4:0}$
7'h33	3'b010	7'h00	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
7'h33	3'b011	7'h00	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)^{unsigned}
7'h33	3'b100	7'h00	xor rd, rs1, rs2	bit-wise XOR	rd = rs1 ^ rs2
7'h33	3'b101	7'h00	srl rd, rs1, rs2	shfit right logical	rd = rs1 >> $rs2_{4:0}$
7'h33	3'b101	7'h20	sra rd, rs1, rs2	shfit right arithmetic	rd = rs1 >>> $rs2_{4:0}$
7'h33	3'b110	7'h00	or rd, rs1, rs2	bit-wise OR	rd = rs1 | rs2
7'h33	3'b111	7'h00	and rd, rs1, rs2	bit-wise AND	rd = rs1 & rs2

Immediate (I-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I-type	imm[11:0]												rs1					funct3			rd					opcode						

opcode	funct3	imm[11:5]	instruction	description	operation
7'h03	3'b000	-	`lb rd, imm(rs1)`	load byte immediate	rd = sign_ext ([address]$_{7:0}$)
7'h03	3'b001	-	`lh rd, imm(rs1)`	load half immediate	rd = sign_ext ([address]$_{15:0}$)
7'h03	3'b010	-	`lw rd, imm(rs1)`	load word immediate	rd = [address]$_{31:0}$
7'h03	3'b100	-	`lbu rd, imm(rs1)`	load byte unsigned	rd = zerp_ext ([address]$_{7:0}$)
7'h03	3'b101	-	`lhu rd, imm(rs1)`	load half unsigned	rd = zero_ext ([address]$_{15:0}$)
7'h13	3'b000	-	`addi rd, rs1, imm`	add immediate	rd = rs1 + sign_ext (imm)
7'h13	3'b001	7'h00	`slli rd, rs1, uimm`	Shift left logical immediate	rd = rs1 << uimm
7'h13	3'b010	-	`slti rd, rs1, imm`	set less than immediate	rd = (rs1 < sign_ext(imm))
7'h13	3'b011	-	`sltiu rd, rs1, uimm`	set less than imm. unsigned	rd = (rs1 < zero_ext(imm))
7'h13	3'b100	-	`xori rd, rs1, imm`	bit-wise XOR immediate	rd = rs1 ^ sign_ext (imm)
7'h13	3'b101	7'h00	`srli rd, rs1, uimm`	Shift right logical immediate	rd = rs1 >> uimm
7'h13	3'b101	7'h20	`srai rd, rs1, uimm`	Shift right arithmetic imm.	rd = rs1 >> uimm
7'h13	3'b110	-	`ori rd, rs1, imm`	bit-wise OR immediate	rd = rs1 | sign_ext (imm)
7'h13	3'b111	-	`andi rd, rs1, imm`	bit-wise AND immediate	rd = rs1 & sign_ext (imm)

Upper Immediate (U-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
U-type	imm[31:12]																				rd					opcode						

opcode	instruction	description	operation
7'h17	`auipc rd, upimm`	add upper immediate to PC	rd = {upimm, 12'b0} + PC
7'h37	`lui rd, upimm`	load upper immediate	rd = {upimm, 12'b0}

RISC-V Instruction Set Summary (2)

Store (S-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S-type	imm[11:5]							rs2					rs1					funct3			imm[4:0]					opcode						

opcode	funct3	instruction	description	operation
7'h23	3'b000	`sb rs2, imm(rs1)`	store byte immediate	$[address]_{7:0} = rs2_{7:0}$
7'h23	3'b001	`sh rs2, imm(rs1)`	store half immediate	$[address]_{15:0} = rs2_{15:0}$
7'h23	3'b010	`sw rs2, imm(rs1)`	store word immediate	$[address]_{31:0} = rs2$

Branch (B-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B-type	[12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				[11]	opcode						

opcode	funct3	instruction	description	operation
7'h63	3'b000	`beq rs1, rs2, label`	branch if equal	If (rs1 == rs2) PC = BTA
7'h63	3'b001	`bne rs1, rs2, label`	branch if not equal	If (rs1 ≠ rs2) PC = BTA
7'h63	3'b100	`blt rs1, rs2, label`	branch if less than	If (rs1 < rs2) PC = BTA
7'h63	3'b101	`bge rs1, rs2, label`	branch if greater than or equal	If (rs1 ≥ rs2) PC = BTA
7'h63	3'b110	`bltu rs1, rs2, label`	branch if less than unsigned	If (rs1 < rs2) PC = BTA
7'h63	3'b111	`bgeu rs1, rs2, label`	branch if greater or equal unsigned	If (rs1 ≥ rs2) PC = BTA

BTA = Branch Target Address, PC = Program Counter

Jump-and-link Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
J-type	[20]	imm[10:1]										[11]	imm[19:12]								rd					opcode						
jalr – I-type	imm[11:0]												rs1					3'b000			rd					opcode						

opcode	instruction	description	operation
7'h67	`jalr rd, rs1, imm`	jump and link register	PC = rs1 + sign_ext(imm), rd = PC + 4
7'h6f	`jal rd, label`	Jump and link	PC = JTA, rd = PC + 4

JTA = Jump Target Address, PC = Program Counter

Register names

name	register number	usage
zero	x0	Constant value 0
ra	x1	Return Addrss
sp	x2	Stack Pointer
gp	x3	Global Pointer
tp	X4	Thread Pointer
t0 – t2	x5 – x7	Temporary registers
s0/fp	x8	Saved register/Frame Pointer
s1	x9	Saved register
a0 – a1	x10 – x11	Function arguments/ return values
a2 – a7	x12 – x17	Function arguments
s2 – s11	x18 – x27	Saved registers
t3 – t6	x28 – x31	Temporary registers

Abbreviations

imm	12-bit signed immediate ($imm_{11:0}$)
uimm	5-bit unsigned immediate ($imm_{4:0}$)
upimm	20-bit upper immediate ($imm_{31:12}$)
address	memory addres: rs1 + sign_ext($imm_{11:0}$)
[address]	data at memory location address
BTA	branch target address: PC + sign_ext({$imm_{12:1}$, 1'b0})
JTA	jump target address: PC + sign_ext({$imm_{20:1}$, 1'b0})
label	text label indicating instruction address
sign_ext	value sign-extended to 32 bits
zero_ext	value zero-extended to 32 bits

SUPPLEMENTARY SHEET FOR QUESTIONS 3 AND 5

You may tear off this page, annotate it with your answer, and attach it to the answer book.



